Stage-3 实验报告

计02 刘明道 2020011156

实验内容

Step-7

首先修改建立符号表的 `frontend.typecheck.namer.visitBlock`：在访问块作用域的子节点时，压入一个新的局部作用域

```
def visitBlock(self, block: Block, ctx: ScopeStack) -> None:
    with ctx.local():
        for child in block:
            child.accept(self, ctx)
```

其中为 `ScopeStack` 实现了上下文管理器的 `__enter__` 和 `__exit__` 。

接下来调整 `backend.dataflow.cfg` ，遍历所有节点，确定从第 0 个节点开始可达的节点集合，并修改相应的迭代器

```
def __init__(self, nodes: list[BasicBlock], edges: list[(int, int)]) -> None:
    ...
    # search the graph and determine nodes reachable from the root
    self.reachable = set()
    q = Queue()
    q.put(0)
    while not q.empty():
        visited_node = q.get()
        self.reachable.add(visited_node)
        for n in self.links[visited_node][1].difference(self.reachable):
            q.put(n)

def iterator(self):
    for n in self.reachable:
        yield self.nodes[n]
```

Step-8

这一节实现对循环语句的支持，具体来说需要参照 `While` 和 `Break` 实现 `For` , `DoWhile` 和 `Continue` 。

AST

首先读到了 `frontend.typecheck.namer` 中相应的提示，照代码中的提示完成了对 `For`， `DoWhile`, `Continue` 节点的访问方法。

但是这些节点还没有定义。于是到 `frontend.ast.tree` 和 `frontend.ast.visitor` 中写好相应 AST 节点及其访问函数在 `Visitor` 中的定义。其中 `For`，`DoWhile` 节点仿照 `While` 实现，`Continue` 节点仿照 `Break` 实现。

Lex & Parse

还需要完成相关节点的解析。首先在 `frontend.lexer.lex.reserved` 中添加相应的保留字

```
reserved = {
    ...,
    "for": "For",
    "do": "Do",
    "continue": "Continue"
}
```

在 `frontend.parser.ply_parser` 中按照规则定义对相应节点的解析

```
def p_for_init(p):
    """
    for_init_elem : opt_expression
    for_init_elem : declaration
    """
    p[0] = p[1]

def p_for(p):
    """
    statement_matched : For LParen for_init_elem Semi opt_expression Semi
opt_expression RParen statement_matched
    statement_unmatched : For LParen for_init_elem Semi opt_expression Semi
opt_expression RParen statement_unmatched
    """
    p[0] = For(p[3], p[5], p[7], p[9])


def p_d_while(p):
    """
    statement_matched : Do statement_matched While LParen expression RParen Semi
    statement_unmatched : Do statement_unmatched While LParen expression RParen
Semi

    """
    p[0] = DoWhile(p[5], p[2])

def p_continue(p):
    """
    statement_matched : Continue Semi
    """
    p[0] = Continue()
```

TACGen

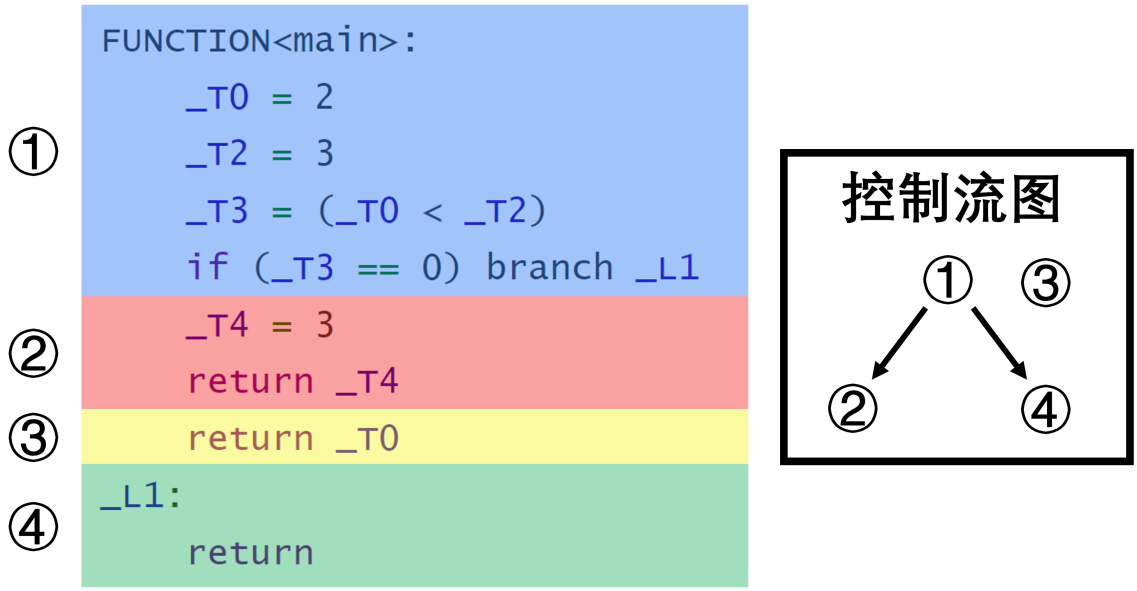
接下来完成三地址码的生成。

- 对于 `Continue` 来说，直接跳转到 `ContinueLabel` 即可。
- 对于 `DoWhile` 来说，可以在 `While` 的基础上增加首次执行时跳过 `cond` 判断的跳转。
- 对于 `For` 来说，执行 `init` 后进入 `cond` - `body` - `update` 的循环，其中`ContinueLabel` 置于 `update` 开始前。
    - `For.cond` 可能为空。因此需要在生成分支语句前进行判断，防止给分支语句赋了空的临时变量。

思考题

Step-7

原 MiniDecaf 代码一种可能的 TAC 码如左图，对应的控制流图见右图。



Step-8

第一种翻译方式在从 `body` 到 `cond` 这一步使用了跳转，而第二种方式则是在开头插入条件判断指令的方式避免了这次跳转。假设循环执行到 `cond` 为假时结束，则对于相同的 `cond` 语句和 `body` ，第二种翻译方式生成的程序每次循环比第一种翻译方式少执行 1 条跳转指令。

因此，第二种翻译方式更好。